

across multiple product boundaries, service layers, and software versions.

Tools selection

The principles that DevOps tool selection is based on are:

- Maximise tool usage and coverage across all services and focus on value.
- Utilise fit-for-purpose contractor tools, or leverage customer tools where required.
- Integration with various sources for analytics.
- Focus on automation and productivity tools for operations.
- Enhance application performance by combining dynamic and static analysis tools.

The DevOps toolchain comprises a set or combination of tools that aid in the delivery, development, and management of applications throughout the system's development lifecycle. It helps to improve productivity, speed up time to market, reduce risks and increase the quality of software.

Seamless cross-tool collaboration is very important in DevOps implementation. An integrated tool system saves a considerable amount of time and effort. Proper connectivity between and across the DevOps toolchain is necessary to create a collaborative work environment around the Dev and Ops teams.

Collaborative development

DevOps supports the interdependence of software development and IT operations, thus helping enterprises to produce software and IT services more rapidly and with frequent iterations. This includes support for polyglot programming, multi-platform development, elaboration of ideas, and creation of user stories, complete with lifecycle management.

Collaborative development includes the practice of continuous integration, which promotes frequent team integrations and automatic builds. By integrating the system more frequently,

integration issues are identified earlier, when they are easier to fix.

Collaborative development has the following benefits:

- Bringing together stakeholders towards a partnered goal.
- Focused on delivering a tangible and functional business outcome.
- Working within a time-boxed scope.
- Using common tools and process platforms.

Source code and software change management

Source code management is used to manage the versions and configurations of assets, merge changes, and manage the integration of changes. Developers can write source code using a code editor to implement the architecture. They need to construct, change and correct applications using a variety of coding models and tools.

Software change management supports software development by providing software configuration control and identification, visibility into the status of software changes, and management of versions of the tools needed to implement the build process.

Build, test and continuous integration

Continuous integration (CI) helps developers to integrate code into a shared repository several times a day. It allows teams to detect problems early and verify each check-in. By integrating regularly, it can detect errors quickly, and locate them more easily.

DevOps host environments may be private, public or hybrid cloud services. Cloud computing is an extremely cost-effective method of deploying, maintaining and securing desktop and server applications.

Containerisation tools

Containerisation tools help in maintaining consistency across the environments where the applications are developed, tested

and deployed. It eliminates failure in a production environment by packaging and replicating the same dependencies and packages used in the development/testing/staging environments.

Repositories

Artifact repository is a collection of binary software artifacts and metadata stored in a defined directory structure. A repository stores two types of artifacts — releases and snapshots. Release repositories are for stable, static release artifacts and snapshot repositories are frequently updated repositories that store binary software artifacts from projects under constant development. As an example, GitHub is the central repository where the code is maintained. Bitbucket and Nexus are the other repository tools.

Continuous release management and deployment

Continuous deployment is wherein every change goes through the pipeline and is automatically put into production, resulting in many production deployments every day with greater delivery speed and frequency for complex applications.

In traditional software development, the changes in code are done quickly, but the reflection of those changes in a production environment takes time, which delays the product release. DevOps can solve this issue by implementing a CI/CD pipeline to speed up development, testing, and releases.

Version controls, updates, and releases are continuously monitored by using DevOps tools. Continuous monitoring across all steps of the application development, testing and deployment is crucial for a successful DevOps implementation. Log management solves the problem of aggregating, storing, and analysing all logs in one place.

The metrics that are relevant and derived from DevOps deployments are: